

Day 29: React Introduction & Setup - Complete Learning Guide

Date: January 13 (Monday)

Focus: Building Your First React Application with True Understanding

Table of Contents

1. Understanding What React Really Is
 2. Setting Up Your Development Environment
 3. Writing Your First Components
 4. Building Your Counter App Project
-

PART 1: CONCEPT FOUNDATION

What Problem Does React Solve?

The Daraz Shopping Cart Story

Imagine you're building the Daraz shopping cart feature using plain JavaScript:

Scenario: A customer adds a product to their cart.

What needs to update on the page?

1. Cart icon badge (showing number of items)
2. Cart total price (in the header)

3. Cart sidebar (showing product list)
4. Product button (changing from "Add to Cart" to "Added")
5. Checkout button (enabling when cart has items)

With Vanilla JavaScript, you'd write:

```
// When user clicks "Add to Cart"
function addToCart(product) {
  // Update 1: Badge count
  document.getElementById('cart-badge').textContent = cartItems.length;

  // Update 2: Total price
  document.getElementById('total-price').textContent = calculateTotal();

  // Update 3: Cart sidebar
  document.getElementById('cart-sidebar').innerHTML = generateCartHTML();

  // Update 4: Product button
  document.getElementById(`btn-${product.id}`).textContent = "Added";

  // Update 5: Checkout button
  document.getElementById('checkout-btn').disabled = false;
}
```

The Problem:

- You have to manually find and update EVERY element
- Miss one? Your UI shows wrong information
- Add a new feature? Update code in multiple places
- Fix a bug? Search through entire codebase

The React Way: Thinking in Data, Not DOM

React's Core Principle: Your UI is a REFLECTION of your data.

Think of it like a **Pakistani railway station display board:**

Old Manual System (Vanilla JS):

- Station master manually flips each board for train arrivals
- If he forgets one board, passengers see wrong info
- Time-consuming and error-prone

New Automatic System (React):

- Station master updates ONE central database
- ALL display boards automatically update
- Impossible to have mismatched information
- Change once, update everywhere

```
// React way: Just change the data
function addToCart(product) {
  setCartItems([...cartItems, product]); // React updates EVERYTHING
automatically
}
```

Pause and Think

Question: In a food delivery app like Foodpanda, when you change the quantity of biryani from 1 to 2, what all needs to update on screen?

Write down at least 5 things that should change:

1. _____

2. _____

3. _____

4. _____

5. _____

Now imagine doing this with `document.getElementById()` for each change. Feel the pain?

vs React vs Vanilla JavaScript: The Mental Model

Understanding the Fundamental Difference

Vanilla JavaScript Approach: Imperative Programming

- You tell the computer HOW to do everything step by step
- Like giving directions: "Go straight, turn left at mosque, turn right at bakery"

React Approach: Declarative Programming

- You tell the computer WHAT you want, it figures out HOW
- Like using Google Maps: "Take me to Packages Mall" - it handles the route

Real Example: Login Button

Vanilla JS (Imperative - "HOW to do it"):

```

const loginBtn = document.getElementById('login-btn');
const spinner = document.getElementById('spinner');
const errorMsg = document.getElementById('error');

function handleLogin() {
  // Step 1: Disable button
  loginBtn.disabled = true;

  // Step 2: Change button text
  loginBtn.textContent = 'Logging in...';

  // Step 3: Show spinner
  spinner.style.display = 'block';

  // Step 4: Hide error
  errorMsg.style.display = 'none';

  // Make API call...
  // Then reverse all these changes based on success/failure
}

```

React (Declarative - "WHAT it should be"):

```

function LoginButton() {
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);

  // Just describe WHAT the UI should look like
  return (
    <button disabled={isLoading}>
      {isLoading ? 'Logging in...' : 'Login'}
    </button>
  );
}

```

Key Differences Comparison

Aspect	Vanilla JavaScript	React
Focus	Manipulating DOM directly	Describing UI based on state
Updates	Manual - you control each change	Automatic - React handles updates
Complexity	Grows with features	Stays manageable
Pakistani Example	Manually updating each price tag in a shop	Computerized billing system that updates everything
Error Prone	Easy to miss updates	Hard to create inconsistent UI
Code Amount	More code for complex features	Less code, more readable

The Virtual DOM: React's Secret Weapon

Understanding Through the Restaurant Analogy

Imagine a Pakistani Restaurant Kitchen:

Traditional Kitchen (Vanilla JS):

- Customer changes order: "No onions in biryani"
- Waiter runs to kitchen
- Chef throws away entire dish
- Starts cooking completely new biryani from scratch
- **Wasteful and slow**

Modern Smart Kitchen (React's Virtual DOM):

- Customer changes order: "No onions"
- Smart system compares new order with current dish
- Chef only removes onions, keeps everything else

- **Efficient and fast**

How Virtual DOM Actually Works

Step 1: React Creates a Virtual Copy

Real DOM (Browser):

```
<div>
  Counter:0
</div>
```

Virtual DOM (React's Memory):

```
<div>
  Counter:0
</div>
```

Step 2: You Update Data (e.g., click counter)

Virtual DOM gets updated FIRST (in memory - super fast):

```
<div>
  Counter:1
</div>
```

← Changed

Step 3: React Compares (Diffing)

Old Virtual DOM

vs

New Virtual DOM

```
<div>
  Counter:0
</div>
```

≠

```
<div>
  Counter:1
</div>
```

↓

Only "0" → "1" needs updating!

Step 4: React Updates Only What Changed (Minimal DOM Manipulation)

Real DOM:
Only the text "0" gets changed to "1"
Not the entire <div> or page!

💡 Why This Matters: The Performance Story

Pakistani Cricket Score Update Example:

When Babar Azam scores a boundary:

- **Without Virtual DOM:** Entire scoreboard refreshes (flickers, slow)
- **With Virtual DOM:** Only the runs number changes (smooth, instant)

In Your Apps:

- Typing in search (like Daraz): Each keystroke is instant
- Social media feeds: Scrolling stays smooth
- Form validation: Instant feedback without page lag

🧩 What is a Component? Building Blocks of React

The Lego Blocks Analogy

Think of building a house with **Lego blocks** vs **molded concrete**:

Molded Concrete (Vanilla JS):

- One giant HTML file
- Change kitchen? Rebuild entire house
- Want same design in another house? Copy-paste everything

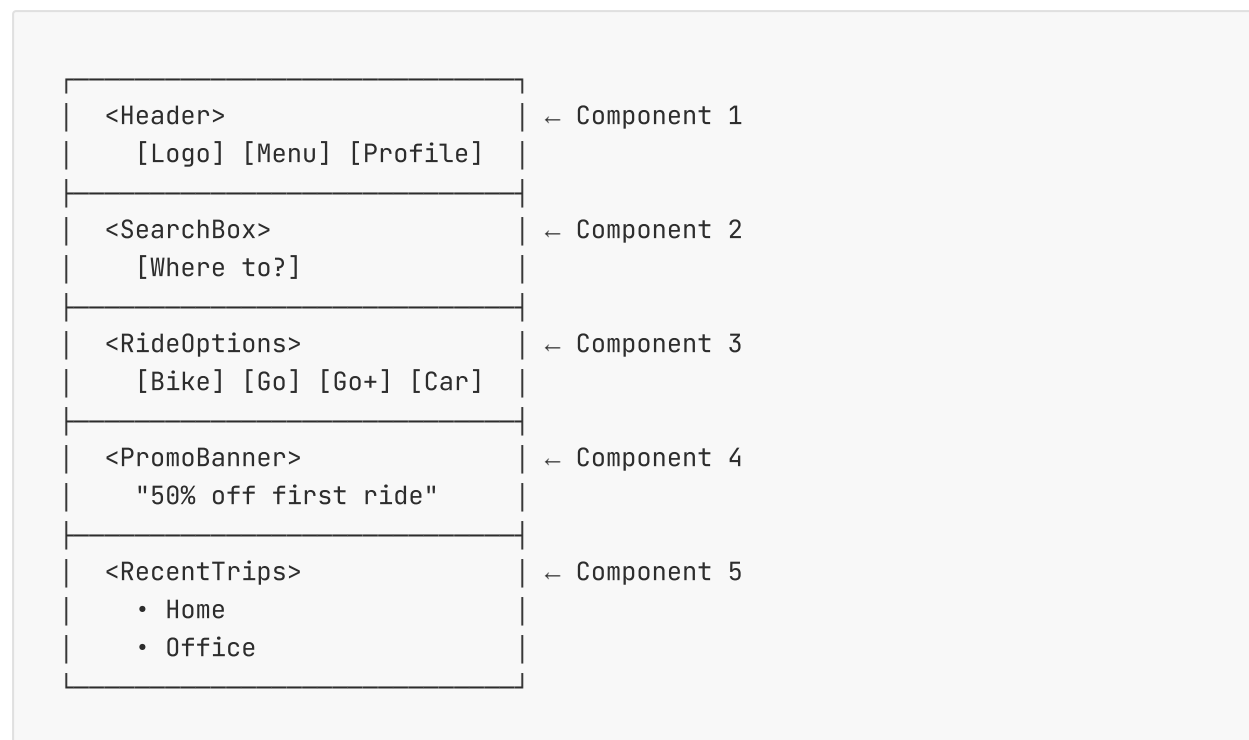
Lego Blocks (React Components):

- Bedroom = One component

- Kitchen = One component
- Bathroom = One component
- Build once, reuse anywhere
- Fix bedroom design, only bedroom changes

Components in Pakistani Apps

Let's Deconstruct the Careem App Screen:



Each component:

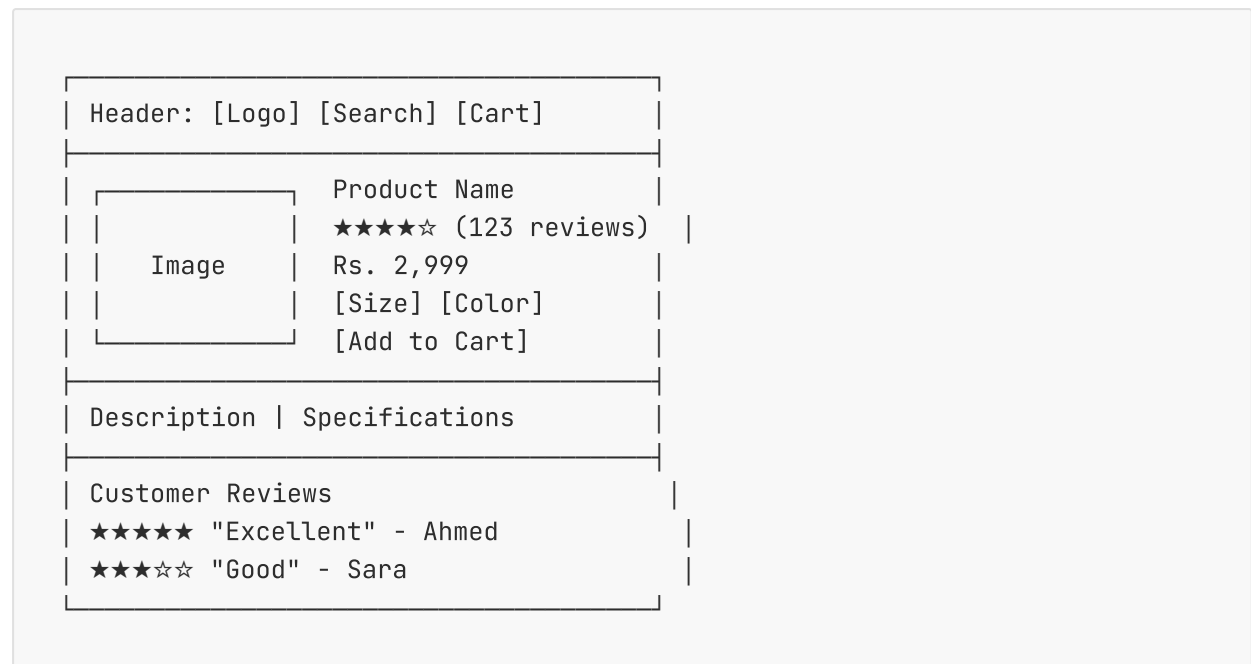
- Has its own job (Single Responsibility)
- Can be reused (Profile in header AND settings page)
- Can be updated independently (Change PromoBanner, nothing else breaks)
- Can have its own data (RideOptions knows selected ride type)

🤔 Checkpoint: Component Thinking

Exercise: Break down a Pakistani e-commerce product page into components.

Daraz Product Page Components:

Identify at least 8 components in this page structure:



List the components:

1. _____
2. _____
3. _____
4. _____
5. _____
6. _____
7. _____

8.

PART 2: FUNDAMENTAL BUILDING BLOCKS

Setting Up Your React Environment with Vite

Why Vite Instead of Create React App?

Think of it like transportation:

Create React App (Old Way):

- Like traveling by train (Rawalpindi to Lahore)
- Comes with EVERYTHING (dining car, sleeper, AC, non-AC)
- Even if you just need a seat for 2 hours
- Heavy, slow to start, lots of unused features

Vite (Modern Way):

- Like traveling by motorway car
- Fast startup (instant ignition)
- Takes only what you need
- Modern, efficient, built for speed

Step-by-Step Setup Process

Prerequisites Check:

Before starting, verify you have:

```
# Check Node.js version (should be 14 or higher)
node --version

# Check npm version
npm --version
```

If not installed: Download from nodejs.org

Creating Your First Vite + React Project

Step 1: Create Project

Open your terminal (Git Bash on Windows, Terminal on Mac/Linux):

```
# Create new Vite project
npm create vite@latest my-first-react-app

# You'll see prompts - choose:
# ? Select a framework: > React
# ? Select a variant: > JavaScript
```

What just happened?

- `npm create vite@latest` = Run Vite's project creator
- `my-first-react-app` = Your project folder name
- Like creating a new folder with all React setup files

Step 2: Enter Project and Install Dependencies

```
# Go into your project folder
cd my-first-react-app

# Install all necessary packages
npm install
```

What is `npm install` doing?

- Reading `package.json` (shopping list of needed packages)
- Downloading React, Vite, and other tools
- Creating `node_modules` folder (like downloading apps)
- Takes 1-2 minutes depending on internet speed

Step 3: Start Development Server

```
# Start the development server
npm run dev
```

You should see:

```
VITE v5.0.0  ready in 500 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

Open browser and go to: `http://localhost:5173/`

You'll see the default Vite + React page!

Understanding Your Project Structure

```
my-first-react-app/
|
├─ node_modules/      ← All installed packages (don't touch)
├─ public/            ← Static files (images, icons)
├─ src/               ← YOUR CODE GOES HERE
|   ├─ App.jsx         ← Main App component
|   ├─ App.css         ← App styles
|   └─ main.jsx        ← Entry point (connects React to HTML)
├─ index.html         ← Single HTML file (React injects here)
├─ package.json       ← Project info & dependencies
└─ vite.config.js     ← Vite configuration
```

Pakistani Shop Analogy:

```
node_modules/    = Warehouse (suppliers' goods)
public/          = Shop display window (visible to all)
src/             = Kitchen/Workshop (where you create)
index.html       = Shop's main entrance
package.json     = Shop's inventory list
```

Exploring Key Files

1. index.html (The Container)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div> ←!— React app will inject here —>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Think of this as:

- The plot of land for your building
- React will construct the entire building in `<div id="root">`

2. src/main.jsx (The Connection Point)

```

import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

// This connects your React app to the HTML
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)

```

Line by line:

- Line 1-4: Import necessary tools and your main App
- Line 7: Find the `<div id="root">` in HTML
- Line 7-8: Create React root there
- Line 8-10: Put your `<App />` component inside
- `<React.StrictMode>`: Developer helper (catches bugs)

3. src/App.jsx (Your First Component)

```

import { useState } from 'react'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <div className="App">
      <h1>Vite + React</h1>
      <button onClick={() => setCount(count + 1)}>
        count is {count}
      </button>
    </div>
  )
}

export default App

```

This is where you'll build!

JSX: JavaScript + HTML = Magic

What is JSX?

JSX = JavaScript XML

It looks like HTML, but it's actually JavaScript that gets converted to HTML.

Think of JSX like Urdu written in Roman script:

- Looks like English (HTML-like syntax)
- But it's actually Urdu (JavaScript)
- Needs translation to be understood (by browser)

JSX vs HTML: The Key Differences

DIFFERENCE 1: `className` INSTEAD OF `class`

HTML:

```
<div class="container">Hello</div>
```

JSX:

```
<div className="container">Hello</div>
```

Why?

- `class` is a reserved word in JavaScript

- Like you can't name your child "if" or "for" in code
- `className` avoids confusion

DIFFERENCE 2: ALL TAGS MUST CLOSE

HTML (Works):

```
<input type="text">
<br>

```

JSX (MUST self-close):

```
<input type="text" />
<br />

```

Think of it like:

- Urdu sentence must end with period (-)
- JSX tag must close with />

DIFFERENCE 3: JAVASCRIPT EXPRESSIONS IN CURLY BRACES

HTML:

```
<p>The sum is 2 + 2</p>

```

The Rule: Anything in `{ }` is evaluated as JavaScript

Examples:

```
const name = "Ahmed";
const price = 500;

return (
  <div>
    <h1>Welcome, {name}!</h1>
    <p>Price: Rs. {price}</p>
    <p>With tax: Rs. {price * 1.17}</p>
    <p>Is expensive? {price > 1000 ? 'Yes' : 'No'}</p>
  </div>
);
```

DIFFERENCE 4: CAMEL CASE FOR ATTRIBUTES

HTML:

```
<button onClick="handleClick()">Click</button>
<label for="name">Name</label>
```

JSX:

```
<button onClick={handleClick}>Click</button>
<label htmlFor="name">Name</label>
```

Pattern: Use camelCase

- `onclick` → `onClick`
- `onchange` → `onChange`
- `for` → `htmlFor`

DIFFERENCE 5: STYLE AS OBJECT

HTML:

```
<div style="color: red; font-size: 20px;">Hello</div>
```

JSX:

```
<div style={{ color: 'red', fontSize: '20px' }}>Hello</div>
```

Notice:

- Double curly braces `{{ }}`
- Outer `{ }` = JavaScript expression
- Inner `{ }` = JavaScript object
- CSS properties in camelCase: `font-size` → `fontSize`
- Values in quotes: `'red'`, `'20px'`

JSX Practice: Build Understanding

Challenge 1: Find the Errors

```
// This code has 5 JSX errors. Find and fix them.
function BrokenComponent() {
  const studentName = "Fatima";
  const marks = 85;

  return (
    <div class="student-card">
      <h1>Student: studentName</h1>
      <p>Marks: {marks}</p>
      <p style="color: green">Status: {marks ≥ 50 ? 'Pass' : 'Fail'}</p>
      
      <input type="text">
    </div>
  )
}
```

Errors to find:

1.

2.

3.

4.

5.

Challenge 2: Convert HTML to JSX

Convert this HTML to valid JSX:

```
<div class="product-card">
  
  <h2>Chicken Biryani</h2>
  <p style="color: red; font-weight: bold;">Rs. 350</p>
  <input type="number" value="1">
  <button onClick="addToCart()">Add to Cart</button>
</div>
```

Your JSX version:

```
function ProductCard() {
  return (
    // Write your converted JSX here
  )
}
```

Creating Your First Component

Component Basics: The Building Block

A React component is a JavaScript function that returns JSX.

Simple Formula:

```
function ComponentName() {
  return (
    // JSX goes here
  );
}
```

Component Naming Rules

Rule 1: Must start with Capital Letter

```
//  Correct
function WelcomeMessage() { }
function UserProfile() { }

//  Wrong
function welcomeMessage() { } // lowercase start
function userprofile() { }    // lowercase start
```

Why? React distinguishes between:

- HTML tags: `<div>`, `<button>` (lowercase)
- Components: `<WelcomeMessage />`, `<UserProfile />` (capitalized)

Rule 2: Use PascalCase (every word capitalized)

```
//  Good names
function StudentCard() { }
function PriceDisplay() { }
function ShoppingCartItem() { }

//  Avoid
function student_card() { } // snake_case
function priceDisplay() { } // camelCase
function shoppingcartitem() { } // all lowercase
```

Rule 3: Be descriptive

```
//  Clear purpose
function ProductCard() { }
function LoginButton() { }
function ErrorMessage() { }

//  Unclear
function Component1() { }
function MyThing() { }
function Stuff() { }
```

Creating Your First Simple Components

Example 1: Static Welcome Component

```
function WelcomeMessage() {  
  return (  
    <div>  
      <h1>Assalam-o-Alaikum!</h1>  
      <p>Welcome to React</p>  
    </div>  
  );  
}
```

Using it in App.jsx:

```
function App() {  
  return (  
    <div>  
      <WelcomeMessage />  
    </div>  
  );  
}
```

Example 2: Component with Data

```
function StudentInfo() {  
  const name = "Ahmed Khan";  
  const rollNumber = "BSCS-2024-001";  
  const marks = 85;  
  
  return (  
    <div className="student-card">  
      <h2>{name}</h2>  
      <p>Roll Number: {rollNumber}</p>  
      <p>Marks: {marks}/100</p>  
      <p>Grade: {marks ≥ 80 ? 'A' : marks ≥ 60 ? 'B' : 'C'}</p>  
    </div>  
  );  
}
```

Breaking it down:

1. Declare variables inside the function
2. Use them in JSX with `{ }`
3. Can do calculations and logic in `{ }`

Component Creation Exercise

Task: Create a Product Card Component

Requirements:

```
function ProductCard() {  
  // TODO: Create variables for:  
  // - productName: "Chicken Biryani"  
  // - price: 450  
  // - restaurant: "Bundu Khan"  
  // - rating: 4.5  
  // - isAvailable: true  
  
  return (  
    <div className="product-card">  
      {/* TODO: Show product name in <h2> */}  
      {/* TODO: Show restaurant in <p> with gray color */}  
      {/* TODO: Show price with "Rs." prefix */}  
      {/* TODO: Show rating with ★ symbol */}  
      {/* TODO: Show "Available" or "Sold Out" based on isAvailable */}  
    </div>  
  );  
}
```

Hints:

- Use template strings for "Rs. 450"
- Use ternary operator for available/sold out
- Unicode for star: `\u2605` or just copy: ★

Test it by adding to App.jsx:

```
function App() {  
  return (  
    <div>  
      <ProductCard />  
    </div>  
  );  
}
```

Understanding Component Return Rules

Rule 1: Must return ONE parent element

✗ This breaks (multiple siblings):

```
function BrokenComponent() {  
  return (  
    <h1>Title</h1>  
    <p>Description</p> // Error: Adjacent JSX elements must be wrapped  
  );  
}
```

✓ **Solution 1: Wrap in div:**

```
function FixedComponent() {  
  return (  
    <div>  
      <h1>Title</h1>  
      <p>Description</p>  
    </div>  
  );  
}
```

✓ **Solution 2: Use Fragment (React.Fragment or <>):**

```
function FixedComponent() {
  return (
    <
      <h1>Title</h1>
      <p>Description</p>
    </>
  );
}
```

When to use Fragment?

- When wrapping div adds unnecessary HTML
- When parent container creates styling issues
- Cleaner DOM structure

Pakistani Example:

```
// Student result card
function ResultCard() {
  return (
    <
      <h2>Semester Results</h2>
      <p>CGPA: 3.8</p>
      <p>Status: Pass</p>
    </>
  );
}
// No extra <div> wrapper in final HTML!
```

Rule 2: Use parentheses for multi-line JSX

✅ **Correct (with parentheses):**

```
function Component() {  
  return (  
    <div>  
      <h1>Hello</h1>  
    </div>  
  );  
}
```

❌ **Incorrect (JavaScript thinks return is on its own):**

```
function Component() {  
  return  
    <div>  
      <h1>Hello</h1>  
    </div>  
;  
}
```

Why? JavaScript automatic semicolon insertion adds `;` after `return`

PART 3: PROGRESSIVE LEARNING PATH 🎓

🧩 Building a Counter App: Step by Step

Stage 1: Understanding State Concept

What is State?

State is data that can CHANGE over time and causes your component to re-render when it changes.

Pakistani Traffic Signal Analogy:

Traffic Signal State:

- Red (Stop)
- Yellow (Get Ready)
- Green (Go)

When state changes: Red → Green

Effect: Cars start moving (UI updates)

In React Counter:

Counter State:

- Could be: 0, 1, 2, 3, 4...

When state changes: 0 → 1

Effect: Display updates (UI shows new number)

Non-State vs State Example

This WON'T work (no state):

```
function BrokenCounter() {  
  let count = 0; // Regular variable  
  
  function increment() {  
    count = count + 1;  
    console.log(count); // This logs, but UI doesn't update  
  }  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={increment}>Increment</button>  
    </div>  
  );  
}
```

Why doesn't it work?

- Regular variables don't trigger re-render
- Like writing on paper vs writing on whiteboard
- Count changes in memory, but React doesn't know to update screen

Introducing useState Hook

useState is like a smart variable that tells React "update the screen when I change"

Basic Syntax:

```
import { useState } from 'react';

function Counter() {
  // Declare state
  const [count, setCount] = useState(0);
  //      [value, setValue]           [initialValue]

  return <p>Count: {count}</p>;
}
```

Breaking down useState:

```
const [count, setCount] = useState(0);
//      |           |           |
//      |           |           └ Initial value (starting count)
//      |           └ Function to update count
//      └ Current value of count
```

Think of it like a bank account:

- `count` = Current balance (read-only)
- `setCount` = Deposit/Withdraw function (only way to change balance)
- `useState(0)` = Opening account with 0 rupees

State Update Rules

Rule 1: Never modify state directly

```
// ❌ WRONG - Don't do this!  
count = count + 1;  
  
// ✅ CORRECT - Use setter function  
setCount(count + 1);
```

Rule 2: State updates are asynchronous

```
setCount(count + 1);  
console.log(count); // Still shows OLD value!  
// React schedules update, doesn't happen immediately
```

Rule 3: State updates trigger re-render

```
setCount(count + 1);  
// Component function runs again  
// JSX returns with NEW count value  
// Screen updates
```

Building Counter - Step 1: Display Only

Task: Create a counter that just displays a number (no buttons yet)

```
import { useState } from 'react';

function Counter() {
  // TODO: Create state variable 'count' with initial value 0
  const [count, setCount] = useState(0);

  return (
    <div className="counter">
      <h1>Counter App</h1>
      { /* TODO: Display the count value */ }
      <p>Current Count: {count}</p>
    </div>
  );
}

export default Counter;
```

Check your understanding:

1. What is the initial value of count? _____
2. What does `setCount` do? _____
3. Can you write `count = 5` to change it? _____

Building Counter - Step 2: Increment Button

Task: Add a button that increases count by 1

```

function Counter() {
  const [count, setCount] = useState(0);

  // TODO: Create increment function
  function handleIncrement() {
    // Use setCount to increase count by 1
    setCount(____);
  }

  return (
    <div className="counter">
      <h1>Counter App</h1>
      <p>Current Count: {count}</p>

      {/* TODO: Add button that calls handleIncrement when clicked */}
      <button onClick={____}>
        Add +1
      </button>
    </div>
  );
}

```

Hints:

- Remember: `setCount(count + 1)` to increment
- `onClick` event takes a function reference
- Don't call the function: `onClick={handleIncrement}` not `onClick={handleIncrement()}`

Test it:

- Click button
- Does number increase?
- Click multiple times
- Does it keep incrementing?

Building Counter - Step 3: Decrement Button

Task: Add decrement button following the same pattern

```

function Counter() {
  const [count, setCount] = useState(0);

  function handleIncrement() {
    setCount(count + 1);
  }

  // TODO: Create handleDecrement function
  function handleDecrement() {
    // Decrease count by 1
  }

  return (
    <div className="counter">
      <h1>Counter App</h1>
      <p>Current Count: {count}</p>

      <button onClick={handleIncrement}>Add +1</button>

      {/* TODO: Add decrement button */}
      <button onClick={___}>
        Subtract -1
      </button>
    </div>
  );
}

```

Building Counter - Step 4: Reset Button

Task: Add button to reset counter to 0

```

function Counter() {
  const [count, setCount] = useState(0);

  function handleIncrement() {
    setCount(count + 1);
  }

  function handleDecrement() {
    setCount(count - 1);
  }

  // TODO: Create handleReset function
  function handleReset() {
    // Set count back to 0
  }

  return (
    <div className="counter">
      <h1>Counter App</h1>
      <p>Current Count: {count}</p>

      <button onClick={handleIncrement}>Add +1</button>
      <button onClick={handleDecrement}>Subtract -1</button>
      {/* TODO: Add reset button */}
    </div>
  );
}

```

Building Counter - Step 5: Prevent Negative

Task: Don't allow count to go below 0

```

function Counter() {
  const [count, setCount] = useState(0);

  function handleIncrement() {
    setCount(count + 1);
  }

  function handleDecrement() {
    // TODO: Only decrease if count is greater than 0
    if (___) {
      setCount(count - 1);
    }
  }

  function handleReset() {
    setCount(0);
  }

  return (
    <div className="counter">
      <h1>Counter App</h1>
      <p>Current Count: {count}</p>

      <button onClick={handleIncrement}>Add +1</button>
      <button onClick={handleDecrement}>Subtract -1</button>
      <button onClick={handleReset}>Reset</button>
    </div>
  );
}

```

Think: What condition checks if count is greater than 0?

Building Counter - Step 6: Visual Feedback

Task: Show different colors based on count value

```

function Counter() {
  const [count, setCount] = useState(0);

  // ... (all your previous functions)

  // TODO: Determine color based on count
  // 0 = gray
  // 1-5 = green
  // 6+ = red
  let countColor;
  if (count === 0) {
    countColor = 'gray';
  } else if (___) {
    countColor = 'green';
  } else {
    countColor = 'red';
  }

  return (
    <div className="counter">
      <h1>Counter App</h1>

      {/* TODO: Apply color to count display */}
      <p style={{ color: ___, fontSize: '48px', fontWeight: 'bold' }}>
        {count}
      </p>

      <button onClick={handleIncrement}>Add +1</button>
      <button onClick={handleDecrement}>Subtract -1</button>
      <button onClick={handleReset}>Reset</button>
    </div>
  );
}

```

Styling Your Counter App

CSS Modules vs Inline Styles vs Regular CSS

Option 1: External CSS File (App.css)

```

/* In App.css */
.counter {
  text-align: center;
  padding: 40px;
  font-family: 'Segoe UI', Arial, sans-serif;
}

.counter-display {
  font-size: 72px;
  font-weight: bold;
  margin: 30px 0;
}

.counter-buttons {
  display: flex;
  gap: 10px;
  justify-content: center;
}

.btn {
  padding: 12px 24px;
  font-size: 18px;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: transform 0.1s;
}

.btn:hover {
  transform: scale(1.05);
}

.btn-increment {
  background-color: #4CAF50;
  color: white;
}

.btn-decrement {
  background-color: #f44336;
  color: white;
}

.btn-reset {
  background-color: #2196F3;
  color: white;
}

```

Using in component:

```
import './App.css';

function Counter() {
  // ... state and functions ...

  return (
    <div className="counter">
      <h1>Counter App</h1>
      <div className="counter-display" style={{ color: countColor }}>
        {count}
      </div>
      <div className="counter-buttons">
        <button className="btn btn-increment" onClick={handleIncrement}>
          Add +1
        </button>
        <button className="btn btn-decrement" onClick={handleDecrement}>
          Subtract -1
        </button>
        <button className="btn btn-reset" onClick={handleReset}>
          Reset
        </button>
      </div>
    </div>
  );
}
```

Option 2: Inline Styles (for dynamic values)

```
// Good for values that change based on state
<div style={{
  color: count > 5 ? 'red' : 'green',
  fontSize: `${count * 10}px` // Grows with count
}}>
  {count}
</div>
```

Styling Challenge

Task: Style your counter to look like a digital calculator

Requirements:

1. Dark background (#2c3e50)
2. Light text color (#ecf0f1)
3. Rounded corners on buttons
4. Different colors for +, -, and reset buttons
5. Add hover effects
6. Make display look like LED screen

Create your styling approach:

```
// You can use external CSS, inline styles, or mix both
// Try to make it look professional and user-friendly
```

Common Mistakes and Debugging

Mistake 1: Calling function instead of referencing

```
// ❌ WRONG - Function gets called immediately on render
<button onClick={handleIncrement()}>Add</button>

// ✅ CORRECT - Pass function reference
<button onClick={handleIncrement}>Add</button>
```

What happens with wrong version?

- Function runs immediately when component renders
- Not when button is clicked
- Can cause infinite loops if it updates state

Mistake 2: Modifying state directly

```
// ❌ WRONG
count = count + 1;
count++;

// ✅ CORRECT
setCount(count + 1);
```

Mistake 3: Forgetting to import useState

```
// ❌ Missing import
function Counter() {
  const [count, setCount] = useState(0); // Error: useState is not defined
}

// ✅ Import first
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // Works!
}
```

Mistake 4: Using wrong quote types for className

```
// ❌ Using backticks unnecessarily
<div className={`counter`} > // Works but unnecessary

// ✅ Regular quotes for static strings
<div className="counter">

// ✅ Backticks for dynamic strings
<div className={`counter ${count > 5 ? 'high' : 'low'}`} >
```

Debugging Checklist

When counter doesn't work:

1. ☒ Is `useState` imported?
2. ☒ Is state being updated with setter function?
3. ☒ Is `onClick` passing function reference (not calling it)?
4. ☒ Are there any console errors? (Check browser console)
5. ☒ Is component exported and imported correctly?

PART 4: INDEPENDENT APPLICATION

Final Project: Enhanced Counter App

Project Requirements

Build a counter app with these features:

Core Features (Must Have):

1. ☒ Display current count value
2. ☒ Increment button (+1)
3. ☒ Decrement button (-1)
4. ☒ Reset button (back to 0)
5. ☒ Prevent negative numbers

Enhanced Features (Choose at least 3):

1. 🎨 Different colors based on count value
2. ➕ Increment by custom amount (e.g., +5, +10)
3. 🎯 Set maximum limit (e.g., can't go above 100)

4.  Show count as progress bar
5.  Display count in different formats (number, word, roman)
6.  Undo/Redo functionality
7.  Responsive design for mobile

Design Thinking Process

Before writing code, answer these:

1. What state do I need?

- Count value (obviously)
- What else? (max value? history? step size?)

2. What events will change state?

- Button clicks
- What else? (keyboard input? double-click?)

3. What validation/logic is needed?

- Prevent negative
- Prevent exceeding maximum
- What else?

4. How will I show feedback to user?

- Color changes
- Disabled buttons when limit reached
- What else?

Starter Code Structure

```
import { useState } from 'react';
import './App.css';

function Counter() {
  // TODO: Declare your state variable(s)

  // TODO: Create your event handler functions

  // TODO: Calculate any derived values (like color, status)

  return (
    <div className="counter">
      {/* TODO: Build your JSX structure */}
    </div>
  );
}

export default Counter;
```

Success Criteria Checklist

Your counter app should:

Functionality:

- ☐ Starts at 0
- ☐ Increases when clicking increment
- ☐ Decreases when clicking decrement
- ☐ Resets to 0
- ☐ Prevents going below 0
- ☐ At least 3 enhanced features working

Code Quality:

- ☐ Uses `useState` correctly
- ☐ Event handlers properly attached

- ☐ No direct state mutations
- ☐ Component exported/imported correctly
- ☐ Console shows no errors

User Experience:

- ☐ Buttons are clearly labeled
- ☐ Visual feedback for different states
- ☐ Responsive to clicks (no lag)
- ☐ Looks professional

Code Organization:

- ☐ Functions have clear names
- ☐ Code is properly indented
- ☐ Comments explain complex logic
- ☐ CSS is organized

Extension Challenges

Challenge 1: Counter with History

Add "Previous" and "Next" buttons to navigate through count history.

Think about:

- How to store history? (array of values)
- How to track current position in history?
- What happens when you change count after going back?

Challenge 2: Multi-Counter

Create multiple independent counters on same page.

Think about:

- Should each counter be a separate component?

- How to make counters reusable?
- How to add/remove counters dynamically?

Challenge 3: Counter with Goal

Set a target number and show progress toward goal.

Think about:

- How to get goal input from user?
- How to calculate percentage progress?
- How to celebrate when goal is reached?

Self-Assessment Questions

After completing your counter, answer these:

Understanding State:

1. What would happen if you used a regular variable instead of `useState`?
2. Why can't you modify state directly (like `count++`)?
3. Explain in your own words what "re-render" means.

Understanding JSX:

1. Why do you use `className` instead of `class`?
2. What's the difference between `onClick={handleClick}` and `onClick={handleClick()}`?
3. How do you display a JavaScript variable in JSX?

Understanding Components:

1. Why must component names start with capital letter?
2. What does it mean that components must return "one parent element"?
3. When would you use `<img />` instead of `<div>`?

Problem Solving:

1. If your button click doesn't update the display, what would you check?
2. How would you add a feature to increase count by 5 instead of 1?
3. How would you make the decrement button disabled when count is 0?

Debugging Guide

If your counter doesn't work:

Problem: Number doesn't update when clicking buttons

Possible causes:

```
// Check 1: Are you calling function instead of passing it?
<button onClick={handleIncrement()}> // ❌ Wrong
<button onClick={handleIncrement}>   // ✅ Correct

// Check 2: Are you modifying state directly?
count = count + 1; // ❌ Wrong
setCount(count + 1); // ✅ Correct

// Check 3: Is useState imported?
import { useState } from 'react'; // Must have this
```

Problem: Getting errors in console

```
// Common errors and fixes:

// "useState is not defined"
// Fix: Add import { useState } from 'react';

// "count is not defined"
// Fix: Declare state: const [count, setCount] = useState(0);

// "Adjacent JSX elements must be wrapped"
// Fix: Wrap multiple elements in <div> or <>

// "className is not defined"
// Fix: Use "className" (string) not {className} (variable)
```

Problem: Styling not applying

```
// Check 1: Is CSS file imported?  
import './App.css';  
  
// Check 2: Is className spelled correctly?  
<div className="counter"> // ✅  
<div classname="counter"> // ❌ Wrong casing  
  
// Check 3: For inline styles, using object?  
style={{ color: 'red' }} // ✅  
style="color: red" // ❌ Not valid in JSX
```

Key Concepts Review

Virtual DOM

Remember: React creates a copy of the DOM in memory, compares changes, and only updates what's different in the real DOM. This makes updates fast and efficient.

Like: Updating only changed items in a shop inventory instead of recreating entire stock list.

Components

Remember: Components are reusable building blocks. They are JavaScript functions that return JSX.

Rules:

- Start with capital letter
- Return one parent element
- Can accept data through "props" (you'll learn this next)

JSX

Remember: JSX looks like HTML but has these differences:

- Use `className` not `class`
- All tags must close (self-close single tags)
- JavaScript expressions go in `{ }`
- Use camelCase for event handlers

State (useState)

Remember: State is data that can change and cause re-render.

Pattern:

```
const [value, setValue] = useState(initialValue);
```

Rules:

- Never modify state directly
- Always use setter function
- State updates are asynchronous
- State updates trigger re-render

Next Steps in Your React Journey

What You've Learned Today

- ✓ Why React exists and what problems it solves
- ✓ How Virtual DOM makes React fast
- ✓ Setting up React project with Vite
- ✓ JSX syntax and rules

- ✅ Creating components
- ✅ Using useState for interactive UIs
- ✅ Building a functional counter app

What's Coming Next

Day 30: Props and Component Communication

- Passing data between components
- Making reusable components
- Component composition

Day 31: Lists and Conditional Rendering

- Mapping arrays to components
- Showing/hiding elements based on conditions
- Keys in lists

Day 32: Forms and User Input

- Controlled components
- Form validation
- Multiple input handling

Practice Recommendations

Today (After this lesson):

1. Build the basic counter (30 minutes)
2. Add 2 enhanced features (30 minutes)
3. Try different styling approaches (20 minutes)
4. Experiment with breaking and fixing code (10 minutes)

This Week:

1. Rebuild counter without looking at notes

2. Create a "Like/Dislike" counter
3. Create a "Prayer Time Tracker" (count completed prayers)
4. Create a "Expense Tracker" (add/subtract money)

Challenge Yourself:

- Build without any errors on first try
 - Explain each line to a friend
 - Teach someone else what you learned
 - Create your own custom counter idea
-

Final Tips for Success

Learning React Effectively:

1. Type, Don't Copy-Paste

- Muscle memory helps learning
- You'll catch errors faster
- Understanding improves

2. Break When Confused

- React has new concepts
- Take breaks to let it sink in
- Review concepts that feel fuzzy

3. Experiment Fearlessly

- Change values and see what happens
- Break things on purpose, then fix them
- Ask "what if I..." and try it

4. Explain to Others

- Best way to solidify learning
- Teaching reveals gaps in understanding
- Use your own words, not documentation

5. Build Small, Build Often

- Don't jump to complex projects
- Master basics first
- Each small win builds confidence

Remember: Every React developer started exactly where you are. The difference between beginners and experts is just practice and persistence.

You've got this! Welcome to React! 🎉

Quick Reference

Import Statement

```
import { useState } from 'react';
```

Component Template

```
function ComponentName() {  
  // State  
  const [value, setValue] = useState(initial);  
  
  // Functions  
  function handleEvent() {  
    // Logic here  
  }  
  
  // Return JSX  
  return (  
    <div>  
      { /* Your JSX */ }  
    </div>  
  );  
}  
  
export default ComponentName;
```

State Pattern

```
const [count, setCount] = useState(0);  
  
// Update state  
setCount(newValue);  
setCount(count + 1);
```

Event Handling

```
<button onClick={functionName}>Click</button>  
<input onChange={handleChange} />  
<form onSubmit={handleSubmit}>
```

Conditional Rendering

```
{condition ? <ComponentA /> : <ComponentB />}  
{condition && <Component />}
```

Inline Styles

```
<div style={{ color: 'red', fontSize: '20px' }}>
```

End of Guide

Created with ❤️ for Pakistani learners embarking on their React journey